

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

Department of Computer Science Engineering

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III / II

Assignment Number: 7.1 / 24

Name: O.Narasimha Raju

HT No: 2303A53034

Batch: 46

Lab Objectives

- Identify syntax, logic, and runtime errors
- Use AI tools for debugging assistance
- Apply systematic debugging strategies
- Improve code reliability and correctness

Task 1: Syntax Error – Missing Parentheses

Buggy Code

```
def greet():  
    print "Hello, AI Debugging Lab!"  
greet()
```

Corrected Code

```
def greet():  
    print("Hello, AI Debugging Lab!")
```

```
greet()
```

```
assert greet() is None
```

Explanation: Python 3 requires parentheses in print statements.

Task 2: Incorrect If Condition

Buggy Code

```
def check_number(n):  
    if n = 10:  
        return "Ten"  
    else:  
        return "Not Ten"
```

Corrected Code

```
def check_number(n):  
    if n == 10:  
        return "Ten"  
    else:  
        return "Not Ten"
```

```
assert check_number(10) == "Ten"  
assert check_number(5) == "Not Ten"  
assert check_number(0) == "Not Ten"
```

Explanation: Single equals (=) is assignment, double equals (==) is comparison.

Task 3: Runtime Error – File Handling

Buggy Code

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()
```

Corrected Code

```
def read_file(filename):  
    try:  
        with open(filename, 'r') as f:  
            return f.read()  
    except FileNotFoundError:  
        return "File not found"
```

```
print(read_file("test.txt"))
print(read_file("missing.txt"))
print(read_file(""))
```

Explanation: Exception handling prevents program crashes.

Task 4: Non-Existent Method

Buggy Code

```
class Car:
    def start(self):
        return "Car started"

my_car = Car()
print(my_car.drive())
```

Corrected Code

```
class Car:
    def start(self):
        return "Car started"

    def drive(self):
        return "Car is moving"

my_car = Car()

assert my_car.start() == "Car started"
assert my_car.drive() == "Car is moving"
```

Explanation: The missing method must be defined or corrected.

Task 5: Type Error

Buggy Code

```
def add_five(value):
    return value + 5

print(add_five("10"))
```

Corrected Code

```
def add_five(value):  
    return int(value) + 5  
  
assert add_five("10") == 15  
assert add_five("5") == 10  
assert add_five("0") == 5
```

Alternative Solution

```
def add_five(value):  
    return str(value) + "5"
```

Explanation: Type conversion resolves mismatch between string and integer.

Conclusion

AI tools help identify and fix bugs quickly.

However, developers must understand the root cause to ensure reliable and maintainable code.